

ICU Mortality Prediction

Modeling Phase: preprocessing, model development, comparison and final test evaluation

Final Test ROC-AUC

0.852

Final Test Recall

0.682

Threshold

0.600

Final candidate

Deep Learning selected due to recall priority.

The model preserved recall-focused behavior on the unseen test set.

1. Preprocessing Strategy

Clean and consistent data preparation for modeling

Final modeling dataset included 45,253 ICU stays.

83 predictors were used: 80 numeric and 3 categorical features.

Categorical features: gender, admission_type, admission_location.

Numeric missing values were filled with median imputation.

Categorical variables were converted with OneHotEncoder.

StandardScaler was used only for scale-sensitive models: Logistic Regression, KNN and Deep Learning.

Categorical features

3

Numeric features

80

Train shape

36,169 x 83

2. Train-Test Split and Leakage Control

Patient-level split for realistic model evaluation

Train/test split was approximately 80/20.

The split was performed at SUBJECT_ID level.

Multiple ICU stays from the same patient were not allowed to appear in both train and test.

The prediction unit was ICU stay, but the split control was patient-level.

SUBJECT_ID and ICUSTAY_ID were not used as model features.

The test set was untouched until final evaluation.

Key idea

Putting the same patient on both sides can let the model “recognize” the patient. Patient-level splitting was therefore a core leakage control step.

3. Modeling and Evaluation Plan

Fair comparison with the same core metrics

Models tested

Logistic Regression
KNN
Decision Tree
Random Forest
XGBoost
Deep Learning

Improvement methods

Hyperparameter tuning
Class weight / scale_pos_weight
SMOTE
Threshold tuning
EarlyStopping

Metrics

ROC-AUC: discrimination power
Precision: accuracy when risk is predicted
Recall: detected true deaths
F1: precision-recall balance

4. Logistic Regression

Baseline model and threshold tuning

Stage	ROC-AUC	Precision	Recall	F1
Baseline	0.842	0.644	0.239	0.349
Balanced	0.843	0.311	0.741	0.438
Selected: threshold 0.2	0.842	0.400	0.570	0.470

Baseline gave good ROC-AUC but very low recall.
class_weight="balanced" increased recall but reduced precision sharply.
Threshold 0.2 on the standard Logistic Regression gave a more recall-focused trade-off.

This model was kept as the reference baseline.

Selected precision

0.400

Selected recall

0.570

Selected F1

0.470

5. KNN

Class imbalance effect in a distance-based model

Stage	ROC-AUC	Precision	Recall	F1
Baseline K=5	0.709	0.614	0.129	0.213
K=3	0.667	0.523	0.167	0.253
SMOTE + K=3	0.739	0.236	0.708	0.353

StandardScaler was used because KNN is distance-based.
Different K values were tested; recall decreased as K increased.
SMOTE increased recall strongly but precision fell sharply.
KNN did not provide a balanced performance comparable to the strongest final candidates.

SMOTE precision

0.236

SMOTE recall

0.708

SMOTE F1

0.353

6. Decision Tree

Tuning and class balancing in a single-tree model

Stage	ROC-AUC	Precision	Recall	F1
Baseline	0.621	0.323	0.339	0.331
Tuned	0.760	0.577	0.142	0.226
Tuned + balanced	0.772	0.253	0.701	0.371
Selected: threshold 0.6	0.772	0.312	0.536	0.394

GridSearchCV controlled excessive tree growth.
Class weight increased recall but precision remained limited.
Threshold 0.6 gave a more acceptable precision-recall balance for this model.

Best parameters

```
max_depth = 5  
min_samples_leaf = 5  
min_samples_split = 2
```

7. Random Forest

More stable results with a tree ensemble

Stage	ROC-AUC	Precision	Recall	F1
Baseline	0.846	0.741	0.159	0.262
Tuned	0.855	0.798	0.135	0.231
Tuned + balanced	0.862	0.455	0.552	0.498

Baseline had good ROC-AUC and precision, but recall was low. Tuning increased precision but reduced recall further. `class_weight="balanced"` meaningfully improved recall and F1.

Best parameters

```
n_estimators = 200
max_depth = None
min_samples_leaf = 5
max_features = sqrt
```

8. XGBoost

Strong precision-recall balance with boosting

Stage	ROC-AUC	Precision	Recall	F1
Baseline	0.855	0.606	0.314	0.413
Tuned	0.870	0.666	0.300	0.414
scale_pos_weight	0.864	0.382	0.680	0.489
Selected: threshold 0.6	0.864	0.446	0.587	0.507

Tuning improved ROC-AUC.
scale_pos_weight gave more importance to the mortality class.
Threshold 0.6 produced the strongest overall balance and the highest F1.

Best parameters

```
n_estimators = 200  
max_depth = 5  
learning_rate = 0.1  
scale_pos_weight ≈ 7.36
```

9. Deep Learning

Final candidate best aligned with recall priority

Stage	ROC-AUC	Precision	Recall	F1
Baseline	0.864	0.624	0.325	0.427
Class weight	0.868	0.350	0.753	0.477
Selected: threshold 0.6	0.868	0.404	0.651	0.499

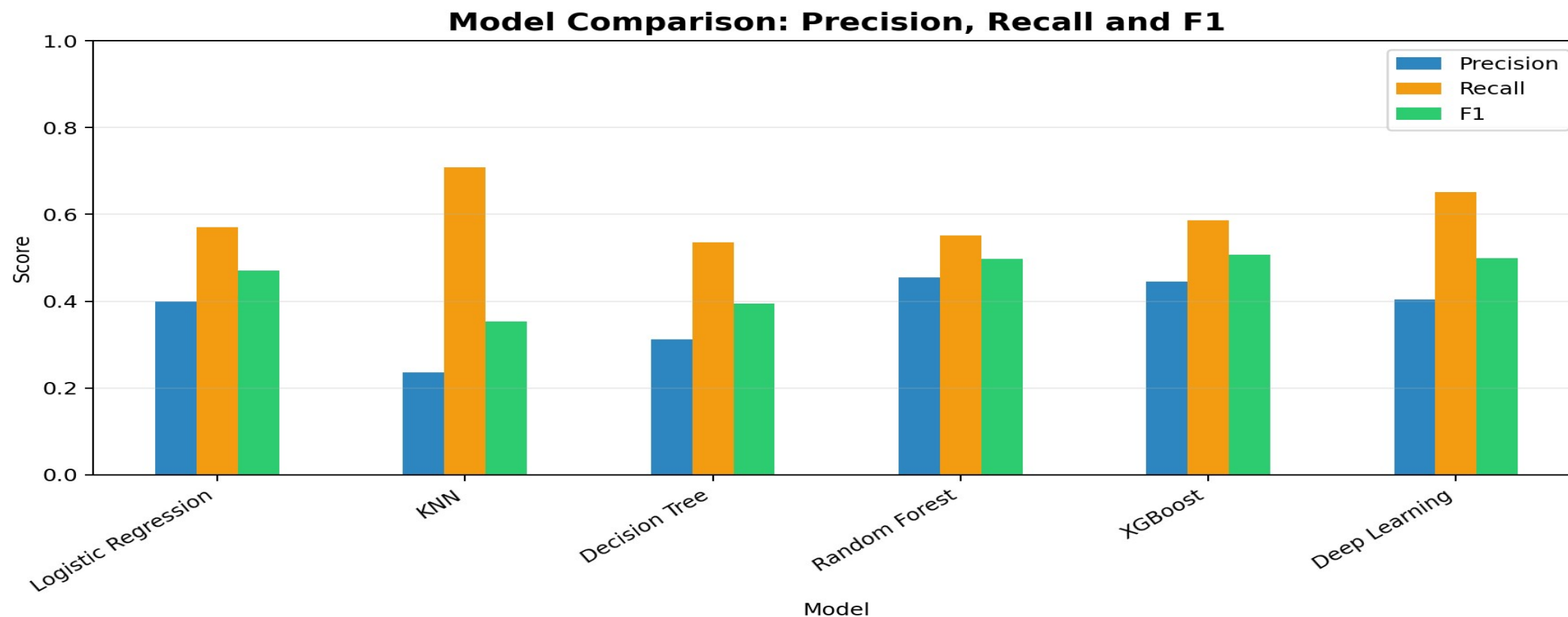
The model was built as a small feed-forward neural network for tabular data.
Class weight strongly improved recall.
Threshold 0.6 was selected as the best recall-focused choice.

Architecture & training

64 -> 32 -> 1
ReLU hidden layers
Sigmoid output
Adam + binary crossentropy
Batch size 64
EarlyStopping

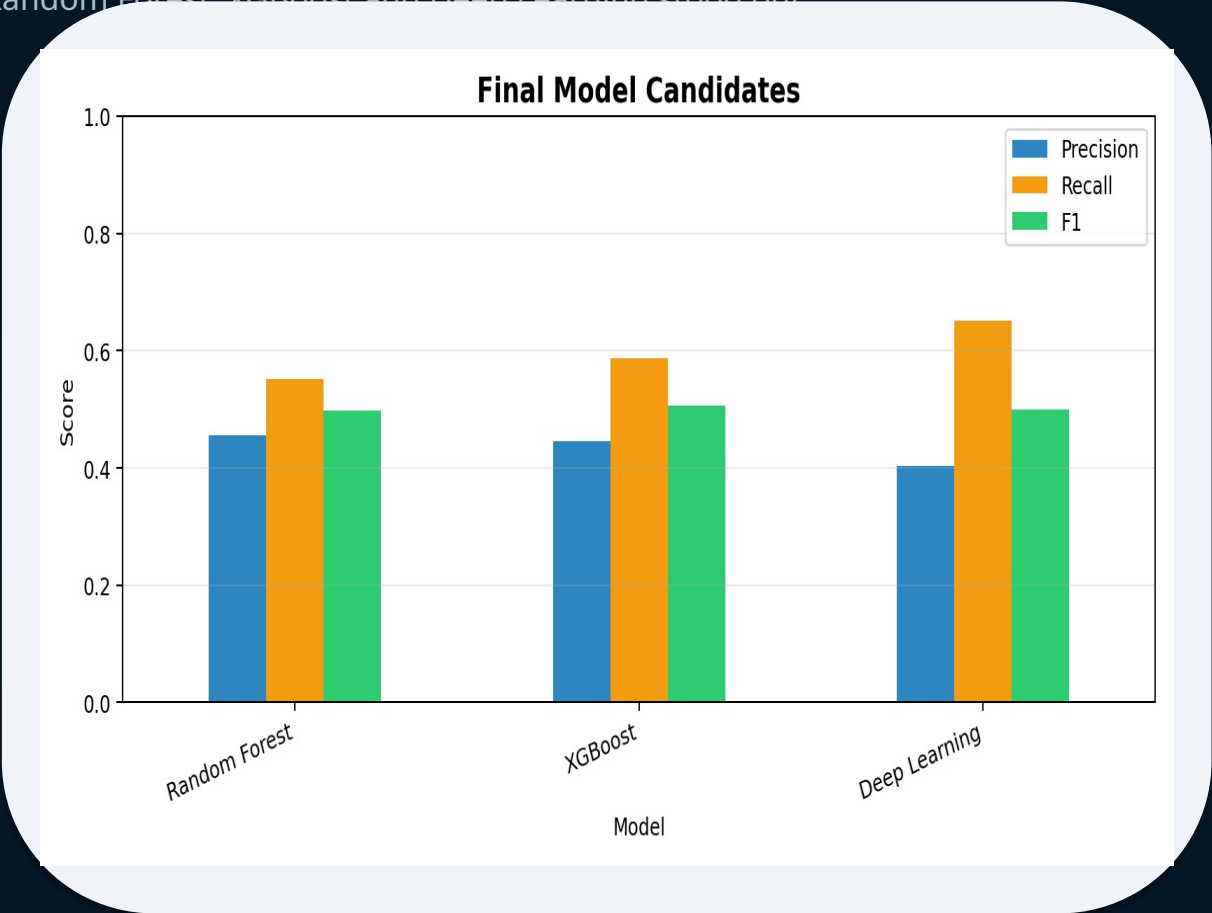
10. All Models Compared

Precision, recall and F1 using the selected best version of each model



11. Final Model Candidates

Random Forest, XGBoost and Deep Learning stood out



Model	ROC-AUC	Precision	Recall	F1
Random Forest	0.862	0.455	0.552	0.498
XGBoost	0.864	0.446	0.587	0.507
Deep Learning	0.868	0.404	0.651	0.499

Random Forest was stronger in precision.
XGBoost gave the most balanced F1 result.
Deep Learning gave the highest recall and best matched the clinical goal.

12. Final Test Evaluation

Selected Deep Learning model evaluated on the untouched test set



The final model was trained on the full train set using class weights.
The validation-selected threshold = 0.6 was used on the test set without change.
The test set was used only at the final evaluation stage.
No dramatic performance drop was observed from validation to test.
The model preserved recall-focused behavior on the test set.

Final decision

Final model: Deep Learning
Main rationale: prioritizing
detection of mortality cases
Final test recall: 0.682